

# **MINI PROJECT REPORT**

on

## **REMOTE DEVICE CONTROL AND MONITORING SYSTEM**

**Computer Networks (CN)**

Submitted by

|                         |                      |
|-------------------------|----------------------|
| <b>Rohit R Naik</b>     | <b>PES2UG24AM217</b> |
| <b>Raksha R Poojary</b> | <b>PES2UG24AM214</b> |
| <b>Prachya</b>          | <b>PES2UG24AM210</b> |

Github repo

<https://github.com/rohitrn936-stack/cnMini.git>

## Abstract

**This project presents the design and implementation of a Remote Device Control and Monitoring System using TCP socket programming in Python. The system establishes a client-server architecture over a local area network, enabling a central server to remotely monitor and issue commands to multiple IoT client devices connected simultaneously. The implementation covers key networking concepts including TCP-based command and control communication, device authentication, status reporting, reliable command execution with acknowledgment, dynamic multi-client management using threading, and latency analysis of command execution. The system successfully demonstrates real-world IoT communication principles without the dependency on specialized embedded hardware, making it reproducible and testable on standard computing machines.**

## Table of Contents

|                                                 |    |
|-------------------------------------------------|----|
| 1. Introduction                                 | 3  |
| 2. Objectives                                   | 3  |
| 3. System Architecture                          | 4  |
| 4. Technologies Used                            | 4  |
| 5. Modules and Features                         | 5  |
| 5.1 TCP-Based Command and Control Communication | 5  |
| 5.2 Device Authentication                       | 5  |
| 5.3 Device Status Reporting                     | 6  |
| 5.4 Reliable Execution and Acknowledgment       | 6  |
| 5.5 Dynamic Multi-Client Management             | 6  |
| 5.6 Latency Analysis                            | 7  |
| 6. Communication Protocol                       | 7  |
| 7. Code Overview                                | 8  |
| 8. Results and Observations                     | 9  |
| 9. Conclusion                                   | 10 |
| 10. References                                  | 10 |

## 1. Introduction

The rapid growth of Internet of Things (IoT) technology has led to the deployment of large numbers of networked devices in homes, industries, and public infrastructure. Managing these devices remotely — issuing commands, retrieving status, and ensuring reliable execution — requires a robust communication mechanism between a central controller and distributed client devices.

This project implements a socket-based Remote Device Control and Monitoring System using the Transmission Control Protocol (TCP). A central server acts as the controller, capable of accepting connections from any number of client devices over a network. Each client device registers with the server, authenticates itself, reports its status, and awaits commands. The server sends control commands, waits for acknowledgment, and records latency metrics.

The implementation is entirely software-based using Python, making it platform-independent. The system can be tested on any standard machines connected to the same network, simulating a real-world IoT deployment without requiring specialized embedded hardware.

## 2. Objectives

- To implement a TCP socket-based server-client communication system in Python.
- To enable a central server to accept and manage connections from a dynamic number of client devices simultaneously.
- To implement an authentication mechanism that validates client identity before permitting command execution.
- To enable real-time device status reporting from clients to the server.
- To ensure reliable command delivery and execution through an acknowledgment protocol.
- To measure and log the latency of command execution for performance analysis.
- To demonstrate cross-platform compatibility — server and clients may run on different operating systems.

## 3. System Architecture

The system follows a client-server architecture. The server runs on one machine and listens for incoming TCP connections on a designated port. Each client device runs on a separate machine and connects to the server using the server's IP address and port number.

**Upon connection, a dedicated thread is spawned on the server for each client. This allows the server to handle multiple client devices concurrently without blocking. The number of clients that can connect simultaneously is not statically limited — any number of client devices can connect dynamically as long as system resources permit.**

**The communication flow for each client session proceeds as follows:**

1. The client establishes a TCP connection to the server.
2. The client sends an authentication token. The server verifies it and either accepts or rejects the connection.
3. Upon successful authentication, the client sends a status report (e.g., online, idle, busy).
4. The server issues a control command to the client.
5. The client executes the command and sends an acknowledgment (ACK) back to the server.
6. The server records the round-trip latency for the command cycle.

## 4. Technologies Used

| Technology / Module       | Purpose                                                    |
|---------------------------|------------------------------------------------------------|
| Python 3.x                | Primary programming language                               |
| socket (Python stdlib)    | TCP socket creation, binding, listening, and data transfer |
| threading (Python stdlib) | Concurrent handling of multiple client connections         |
| time (Python stdlib)      | Timestamp recording for latency measurement                |
| TCP/IP Protocol           | Reliable, connection-oriented network communication        |
| Local Area Network (LAN)  | Network infrastructure connecting server and clients       |

## 5. Modules and Features

### 5.1 TCP-Based Command and Control Communication

**The core of the system is built on the Python socket library using TCP (SOCK\_STREAM). TCP was chosen over UDP because it provides:**

- Connection-oriented communication with a guaranteed session between server and client.
- Reliable, ordered delivery of data packets.
- Built-in error detection and retransmission.

The server creates a socket, binds it to the address 0.0.0.0 (accepting connections on all available network interfaces) and a port number, and listens for incoming connections. Each client creates a socket and connects to the server's IP address and port. Once connected, both ends communicate by sending and receiving encoded byte strings.

## 5.2 Device Authentication

**Before any command is issued, the server requires each connecting client to authenticate itself. Authentication is implemented as follows:**

7. Immediately after connection, the client sends a message of the format AUTH:<token>.
8. The server checks the received token against a pre-configured valid token.
9. If the token matches, the server responds with AUTH:success and proceeds with the session.
10. If the token does not match, the server responds with AUTH:failed and closes the connection immediately.

This prevents unauthorized devices from receiving or executing commands. In a production system, this mechanism can be extended to certificate-based or challenge-response authentication.

## 5.3 Device Status Reporting

**After successful authentication, the client sends a status report to inform the server of its current operational state. The status message follows the format STATUS:<state>, where <state> may be one of online, idle, or busy.**

The server receives and logs this status before dispatching any command. This allows the server to maintain awareness of each device's condition and can be extended to support conditional command dispatch based on device state.

## 5.4 Reliable Execution and Acknowledgment

**Every command sent by the server follows the format CMD:<command\_name>. Upon receiving a command, the client executes the corresponding action and sends back an acknowledgment message in the format ACK:<result>.**

The server waits for the ACK before proceeding, ensuring that commands are not dispatched in an uncontrolled manner. This request-acknowledge cycle guarantees reliable command execution and provides a foundation for building more complex command pipelines with retry logic.

## 5.5 Dynamic Multi-Client Management

**The server uses Python's threading module to manage multiple client connections concurrently. When the server accepts a new connection, it immediately spawns a new thread assigned to handle that client session independently. This design means:**

- The server is never blocked by a single slow or unresponsive client.
- Any number of clients can connect and be served simultaneously, limited only by available system resources.

- Each client thread operates independently, maintaining its own authentication, status, and command cycle.

This approach scales dynamically — there is no hard-coded limit on the number of connected clients. The server continues to accept new connections indefinitely as long as it is running.

## 5.6 Latency Analysis

**Latency analysis measures the round-trip time (RTT) of a command execution cycle. The server records a timestamp (using `time.time()`) immediately before sending a command and another immediately after receiving the corresponding ACK. The difference, expressed in milliseconds, represents the time taken for the command to travel to the client, be executed, and the acknowledgment to return.**

**This metric is logged to the console for each command-ACK cycle and can be used to evaluate network performance, detect slow clients, or identify network congestion. On a local area network, typical latency values observed are below 20 milliseconds.**

## 6. Communication Protocol

**All messages exchanged between the server and clients follow a simple text-based protocol using colon-separated key-value pairs. Messages are encoded as UTF-8 byte strings before transmission and decoded upon receipt.**

| Direction       | Message Format | Example           | Meaning                                    |
|-----------------|----------------|-------------------|--------------------------------------------|
| Client → Server | AUTH:<token>   | AUTH:securekey123 | Client sends authentication token          |
| Server → Client | AUTH:success   | AUTH:success      | Authentication accepted                    |
| Server → Client | AUTH:failed    | AUTH:failed       | Authentication rejected; connection closed |
| Client → Server | STATUS:<state> | STATUS:online     | Client reports its current status          |
| Server → Client | CMD:<command>  | CMD:shutdown      | Server issues a control command            |
| Client → Server | ACK:<result>   | ACK:done          | Client confirms command execution          |

## 7. Code Overview

### 7.1 server.py

**The server script performs the following operations in sequence:**

11. Imports the socket, threading, and time modules.
12. Creates a TCP socket and binds it to 0.0.0.0 on the designated port, enabling it to accept connections from any network interface.
13. Enters an infinite loop, accepting client connections. For each new connection, a new thread is created and started, calling the `handle_client()` function.
14. `handle_client()` manages the full lifecycle of a single client: authentication check, status receipt, command dispatch, ACK receipt, and latency logging.

**Key code structure:**

```
import socket, threading, time

def handle_client(conn, addr):
    # Step 1: Authenticate
    auth = conn.recv(1024).decode()
    if auth != 'AUTH:securekey123':
        conn.send('AUTH:failed'.encode())
        conn.close(); return
    conn.send('AUTH:success'.encode())
    # Step 2: Receive status
    status = conn.recv(1024).decode()
    # Step 3: Send command and measure latency
    start = time.time()
    conn.send('CMD:turn_on'.encode())
    ack = conn.recv(1024).decode()
    latency = (time.time() - start) * 1000
    print(f'Latency: {latency:.2f} ms')

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('0.0.0.0', 9999))
server.listen()
while True:
    conn, addr = server.accept()
    threading.Thread(target=handle_client, args=(conn, addr)).start()
```

**7.2 client.py****The client script performs the following:**

15. Creates a TCP socket and connects to the server's IP address and port.
16. Sends the authentication token.
17. If authentication succeeds, sends a status report.
18. Waits for a command from the server, executes it, and sends back an ACK.

**Key code structure:**

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('<server_ip>', 9999))

# Authenticate
client.send('AUTH:securekey123'.encode())
response = client.recv(1024).decode()
if response != 'AUTH:success': exit()
```

```
# Send status
client.send('STATUS:online'.encode())

# Receive command and acknowledge
cmd = client.recv(1024).decode()
print(f'Executing: {cmd}')
client.send('ACK:done'.encode())
```



## 8. Results and Observations

The system was tested with multiple client devices connected to a server over a local area network. All machines ran on the same WiFi network. The following observations were recorded:

| Parameter                                | Observed Value / Result                                          |
|------------------------------------------|------------------------------------------------------------------|
| Number of simultaneous clients           | Dynamic (tested with multiple clients; no upper limit imposed)   |
| Transport protocol                       | TCP (SOCK_STREAM)                                                |
| Average command round-trip latency (LAN) | < 20 ms                                                          |
| Authentication — valid client            | AUTH:success received; session proceeds                          |
| Authentication — invalid client          | AUTH:failed received; connection closed immediately              |
| Command acknowledgment                   | ACK:done received for every command dispatched                   |
| Cross-platform compatibility             | Verified — server and clients ran on different operating systems |
| Server blocking on single client         | Not observed — threading ensures concurrent handling             |

The server correctly handled all simultaneous connections without blocking. Each client was served by an independent thread, and latency measurements remained consistently low throughout the test. Authentication successfully rejected clients that provided incorrect tokens, and all valid clients received and acknowledged commands without errors.

## 9. Conclusion

This project successfully demonstrates the design and implementation of a Remote Device Control and Monitoring System using TCP socket programming in Python. The system addresses the core requirements of IoT remote management: secure authentication, real-time status reporting, reliable command execution with acknowledgment, and latency measurement.

The use of Python's threading module enables the server to manage a dynamic and unbounded number of client connections concurrently, making the architecture scalable without any static configuration. The system is platform-independent and has been

**verified to operate correctly across different operating systems connected on the same network.**

**The project reinforces fundamental Computer Networking concepts including the TCP/IP protocol stack, socket programming, the three-way handshake, and client-server communication models. Future enhancements could include encrypted communication using TLS, a persistent logging mechanism, a graphical monitoring dashboard, and integration with actual embedded hardware devices.**